LAB4
Manage Recurring Bills APP

Develop a Flutter application that helps users manage recurring bills such as electricity, water, rent, internet, insurance, and subscriptions.
The application must allow users to:

- Add new bills
- Update existing bills
- Delete bills
- Mark bills as Paid or Unpaid
- View all bills
- View a summary of unpaid bills

All data must be stored locally using **SQLite (sqflite package)**.

Data Structure

Each bill includes the following attributes:

Field	Type	Description
`id`	integer	Unique identifier (auto-incremented)
`bill_name`	text	Bill name (e.g., "Electricity - September")
`category`	text	Type of bill (e.g., "Utilities", "Subscription", "Rent", "Finance", "Health")
`amount`	real	Amount in VND (e.g., 850.000)
`status`	text	Either "Paid" or "Unpaid"
`due_date`	text	Due date in ISO format (e.g., "2025-10-30")

Requirements 1: Bills List Screen (Home Screen)

Create the main screen displaying all recurring bills *(see **Figure 1**)*. This screen must include:

- Display bills in a **FlatList** showing: Bill Name, Category, Amount, Due Date, and Status.

- Use **Bottom Tab Navigator** to switch between screens.
- On first launch, the list may be empty.
- Display an **Add FloatingActionButton** for creating a new bill.
- Show a **banner image** at the top of the Home screen.
- Tapping a bill navigates to its **Update screen**.

Flutter Widgets Suggested

- Scaffold
- AppBar
- Image.asset()
- ListView.builder()
- Card
- ListTile
- FloatingActionButton

Task 2: Add New Bill Screen

Create a screen to add new bills (*see **Figure 3***):

- Input fields: Bill Name, Category, Amount, Due Date, and Status.

Note: The id field is auto-generated, and status defaults to "**Unpaid**".
After pressing **Save**:

- Validate all input.
- Insert the bill into SQLite.
- Return to Home Screen.
- Refresh the bill list automatically.

Provide a Back button to return without saving.

Flutter Widgets Suggested

- Form
- TextFormField

- DropdownButtonFormField
- DatePicker
- ElevatedButton
- Navigator.pop()

Requirements 3: Update Bill Screen

- Create a screen for updating an existing bill. Allow users to **modify** Bill Name, Category, Amount, Due Date, or Status, and **save changes** to Database *(see **Figure 4**)*:
- Display the selected bill's current information in editable input fields.
- Include three buttons on the same row:
    - Update: Save modified data and return to Home screen.
    - Delete: Permanently remove the bill.
    - Back: Return to Home screen

Flutter Widgets Suggested

- AlertDialog
- Row
- ElevatedButton
- Navigator.pop()

Requirements 4: Unpaid Bills Summary Screen

Create a screen that displays only bills with status = "Unpaid" and shows a summary *(see **Figure 5**)*:

- Display all unpaid bills using a FlatList (showing name, category, and amount).
- Calculate and display at the bottom: the total outstanding amount (sum of all unpaid bills).

Flutter Widgets Suggested

- BottomNavigationBar
- ListView.builder
- Column
- Text

Requirements 5: Additional Requirements

- Validate input fields and provide immediate feedback for adding, updating, and deleting bills, including confirmation before deletion (*see **Figure 2***).
- UI/UX: Clean, modern interface with consistent colors, icons, and spacing.

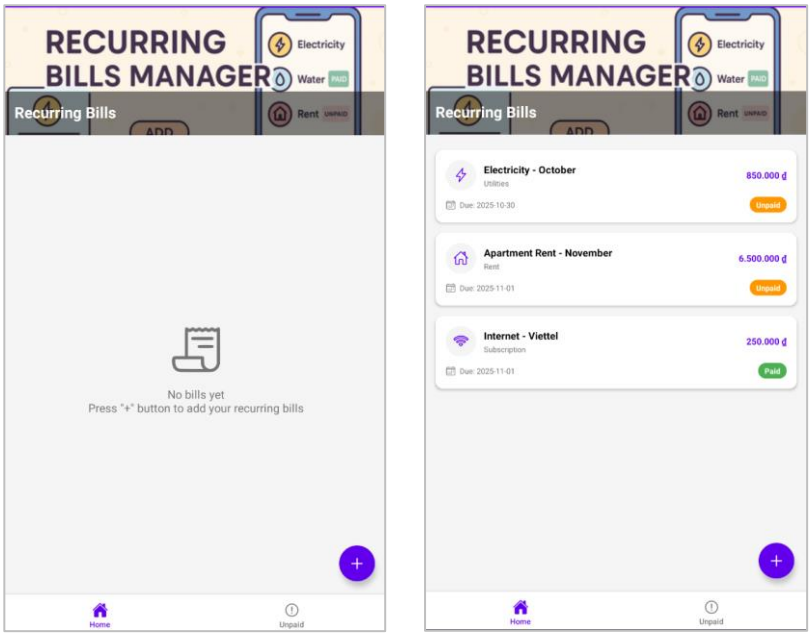


Figure 1: *Home Screen before and after adding recurring bills.*

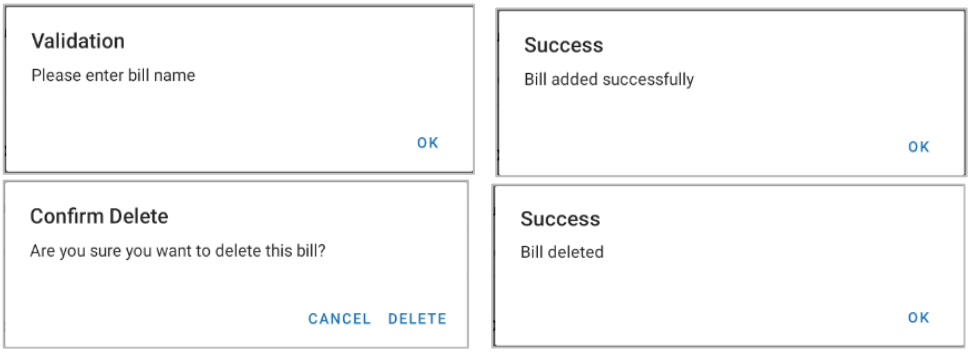


Figure 2: *Screens showing input validation (empty bill name), delete confirmation dialog, and success messages for add, update, and delete operations.*

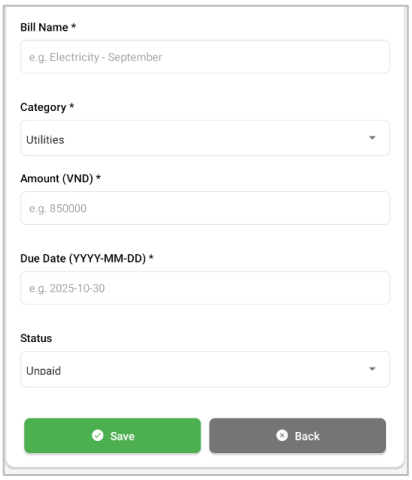


Figure 3: *Add Screen for creating a new recurring bill, with input fields, validation, and a submit button.*

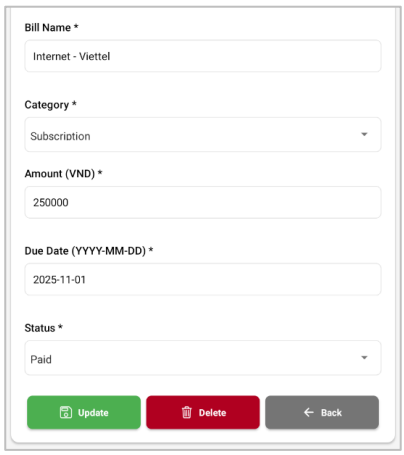


Figure 4: *Update Screen for editing bill information with fields for modification, and buttons to update or delete the bill.*

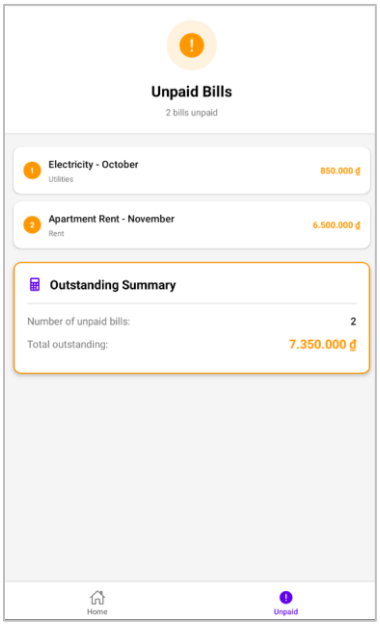


Figure 5: *Summary Screen – shows all unpaid bills and displays the total outstanding amount at the bottom.*